DLL Malware Emotet

What is a DLL (Dynamic Link Library)?

A **DLL** is a library that contains code and data that can be used by more than one program at the same time. In the Windows ecosystem, it is an implementation of the **shared library** concept.

Unlike an executable (`.exe`), a DLL cannot be run directly by the user. It must be loaded by another process (a "host") to execute its instructions.

Executive Summary

The sample is a **position-independent Windows shellcode** that acts as an **obfuscated loader**. It has no import table and resolves all Windows APIs at runtime via the Process Environment Block (PEB). Its purpose is to load a DLL (whose name is stored encrypted), resolve at least one export from that DLL, decrypt two embedded blobs (likely payload or configuration), and execute a multi-step state machine that coordinates execution and cleanup.

Sample Identification

Property	Value
File Name	bad.dll
File Type	Win32 DLL (PE32 executable, GUI, Intel 80386)
Main Export	`Control_RunDLL` at `0x10001070`
File Size	252.50 KB (258,560 bytes)
MD5	`30891d20c065fe6378c9b2568c73a7f0`
SHA-1	`76fffb0ef26bacf0925de63886cf12e911e8793a`
SHA-256	`3897dbbf619853c8f37abb9e653487ea12a38b1f1e2d02d5bbd3ccf8e4e4a8a5`
Imphash	`c50e47fa2c7197441952918ce6851ec0`
Vhash	`125046756d156az45?z1`
Authentihash	`2742183995f2520427b9533a7611c5c66c9d5eec3f639483ffc109943b0b4d0f`
Rich Header Hash	`d0e0ef099437af678e3b1abb413706f5`
SSDEEP	`3072:PtgItJoMl9eJ02kGuBDhk3VsbwVBQdP6ZkiaoZa74jZUUzdDIm6O80MTcdfokHJm:OHK9eSBFA+bwVB35tMTc5ocEFWTBiz`
TLSH	`T10944BF00B280A072D9FF193A45E5C6694ABC7A500F90D9CF639858BE5F775C2B6309EF`
Compiler	Microsoft Visual C/C++ (19.29.30137) [VS 2019 v16.11]
Linker	Microsoft Linker (14.29.30137)
Magika/DIE	PEBIN / PE32

PE section

Section	Full SHA-256 Hash	Entropy	Raw Size	Virtual Size
.text	`F375265F8DE15EE8DF393271A266A3E5149AC8A3B9769555F8CC9756D3EFE2D2`	7.491	208,896	208,644
.rdata (0)	`3917F41EBCFCA3C420A297C2B0DA4EF610666313B37B2ECBFFDDADA4C66471DE`	6.024	39,936	39,698
.rdata (1)	`EB58E9C9894EA94E2068D4721EF303FAB5729473DDD1450067D830A9A29842C9`	6.024	3,584	5,936
.data	`D14EDFA50DDAE33D8A51BC7C97431D30B4379D383BE9EE50F7F321BF54DB645C`	3.295	5,120	4,980

Emotet DLL Import Table (KERNEL32.dll)

Function Name	Address (IAT)	Type	Significance in Malware Analysis

VirtualAlloc	`0x0003D60C`	Memory	**High Alert.** Used to allocate a new buffer in RAM to store the decrypted/unpacked payload.
VirtualProtect	`0x0003D61C`	Memory	**High Alert.** Used to change memory permissions to `EXECUTE`, allowing the unpacked code to run.
GetProcAddress	`0x0003D62E`	Dynamic Loading	Used to find the address of other functions. Helps the malware hide its full list of imports.
LoadLibraryA / ExW	`0x0003D640`	Dynamic Loading	Loads additional DLLs (like networking or crypto modules) at runtime to stay modular.
IsDebuggerPresent	`0x0003D6C6`	Anti-Analysis	Checks if you are running it in x64dbg. If true, it might crash or behave differently to fool you.
CreateFileW	`0x0003DAE4`	File System	Likely used for dropping a second stage or creating a persistence file on disk.
WriteFile	`0x0003DA72`	File System	Writes the malicious payload or configuration data to a file.
TerminateProcess	`0x0003D76A`	Control	Can be used to kill security software or shut down if a debugger is detected.
TlsAlloc / TlsFree	`0x0003D862`	Execution	Thread Local Storage; often used by malware for anti-debugging or to store shellcode.
GetTickCount / QPC	`0x0003D650`	Anti-Analysis	Emotet uses timing checks (QueryPerformanceCounter) to see if it's being "traced" slowly by an analyst.

Export

Ordinal	Hint	Function Name	Entry Point (RVA)	Source Section
1	1	**Control_RunDLL**	`0x00001070`	`.text`

Execution: [LIVING OFF THE LAND]

- **Command:** `rundll32.exe <sample>.dll,Control_RunDLL`
- **Tactic:** Uses the `Control_RunDLL` export to masquerade as a legitimate Windows Control Panel process, evading basic sandbox triggers.

Description

Classification: Win32 DLL | **Family:** Emotet | **Hash (SHA-256):** `3897dbbf619853c8f37abb9e653487ea12a38b1f1e2d02d5bbd3ccf8e4e4a8a5`

- **Packing Status: YES, HEAVILY PACKED.**
  - **Evidence:** The `.text` section exhibits a high entropy of **7.491**, signaling encrypted code. Additionally, the `.rdata` section's Virtual Size is significantly larger than its Raw Size, a classic indicator of memory reservation for an unpacked payload.
- **Execution:** Launches via the `Control_RunDLL` export using `rundll32.exe`, mimicking legitimate Windows Control Panel operations.
- **Behavior:** Features a "bootstrapping" toolkit including `VirtualAlloc` and `VirtualProtect` to unpack its core payload into memory, alongside `IsDebuggerPresent` to evade analysis in debuggers or virtual machines.

Analysis of the DLL

Executing the DLL

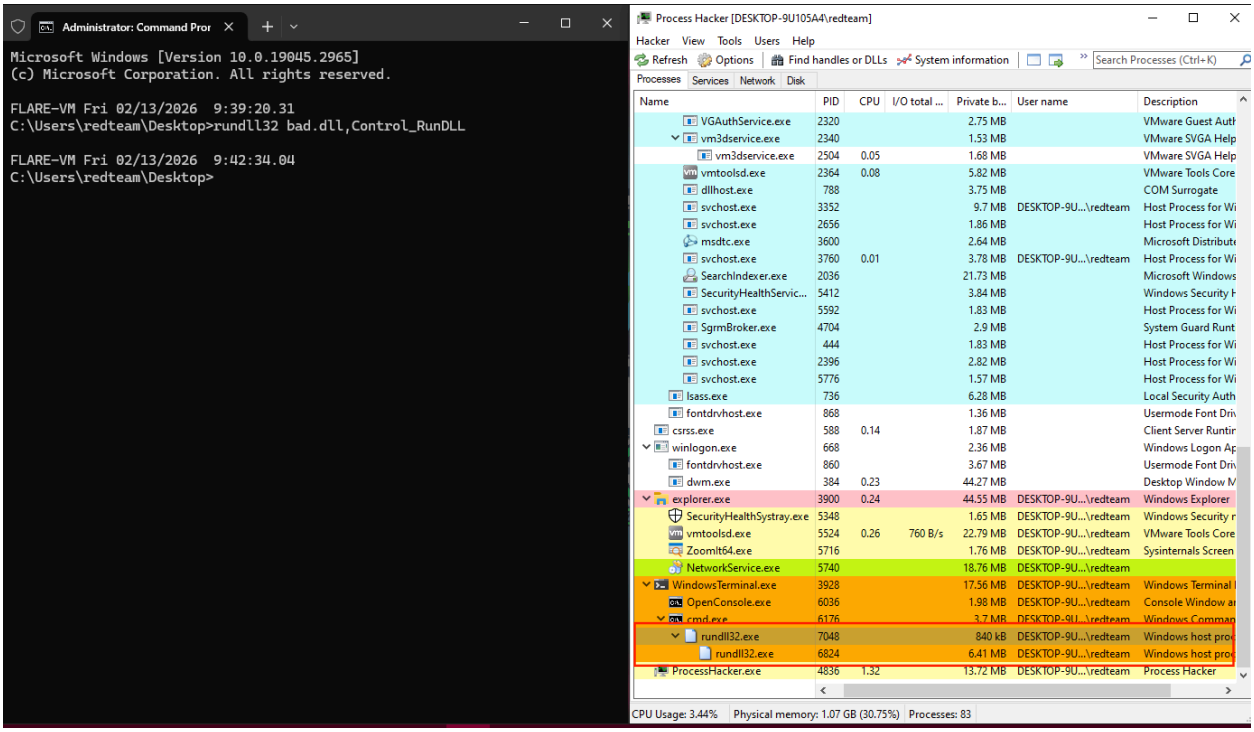

Fig 1 - Executing the dll monitoring with process hacker

Network Activity

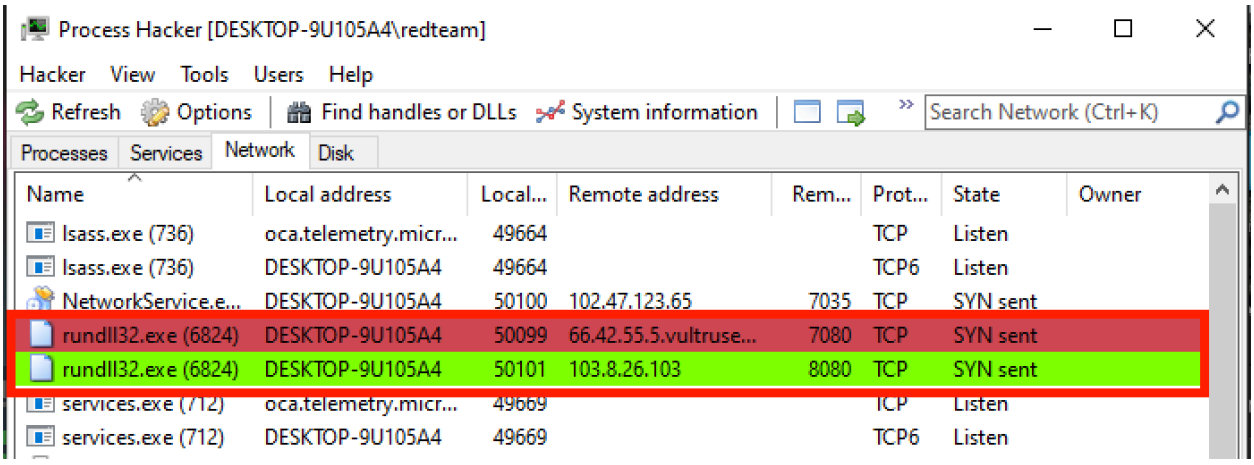

Fig 2 - Network activity by dll

Description

After executing the DLL via `rundll32.exe`, the process was monitored using **Process Hacker 2**. The following outbound network activity was observed, signaling a "Beaconing" phase where the malware attempts to contact its Command & Control (C2) infrastructure.

Ghidra Analysis

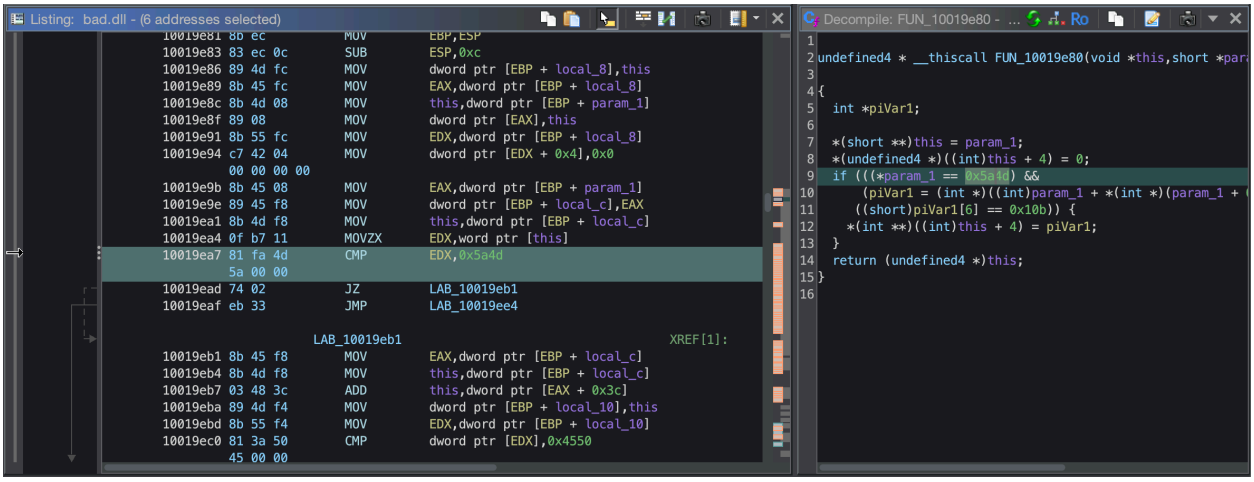

Fig 3 - Validating PE header

Code Snippet	Technical Explanation
`*(short **)this = param_1;`	Stores the pointer to the memory buffer (`param_1`) into the object's first field.
`*param_1 == 0x5a4d`	**MZ Header Check:** Checks if the first two bytes are `4D 5A` ("MZ" in ASCII). This is the signature for all DOS/Windows executables.
`param_1 + 0x1e`	**e_lfanew pointer:** Accesses the offset to the "PE" header. In hex, `0x3C` (which is `0x1E` shorts) is where the offset to the NT headers is stored.
`*piVar1 == 0x4550`	**PE Signature Check:** Checks if the signature at that offset is `50 45 00 00` ("PE\0\0").
`(short)piVar1[6] == 0x10b`	**Magic Number Check:** `0x10B` indicates this is a **PE32** (32-bit) executable. (0x20B would be 64-bit).
`*(int **)((int)this + 4) = piVar1;`	If all checks pass, it stores the pointer to the validated PE header into the object's second field.

Function `FUN_10019e80` **:** This sub-routine acts as a **PE Header Parser**. It validates that a specific memory region contains a valid Win32 executable by verifying the **MZ (DOS)** signature and the **PE (NT)** signature. This is a critical step in the malware's self-unpacking process, ensuring the decrypted payload is structurally sound before transferring execution control.

Function call References

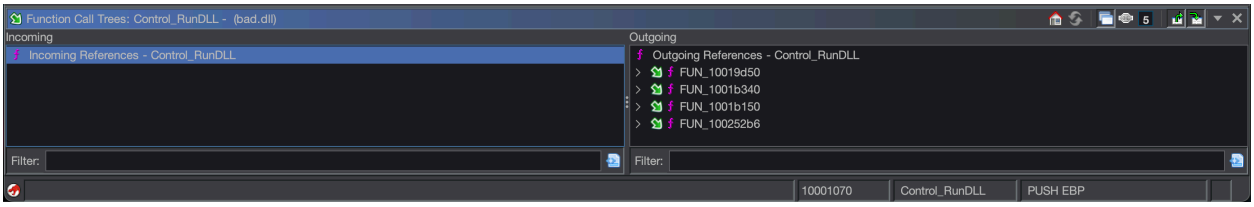

Fig 4 - ghidra function call references

X32DBG

load rundll32.exe first in x32dbg

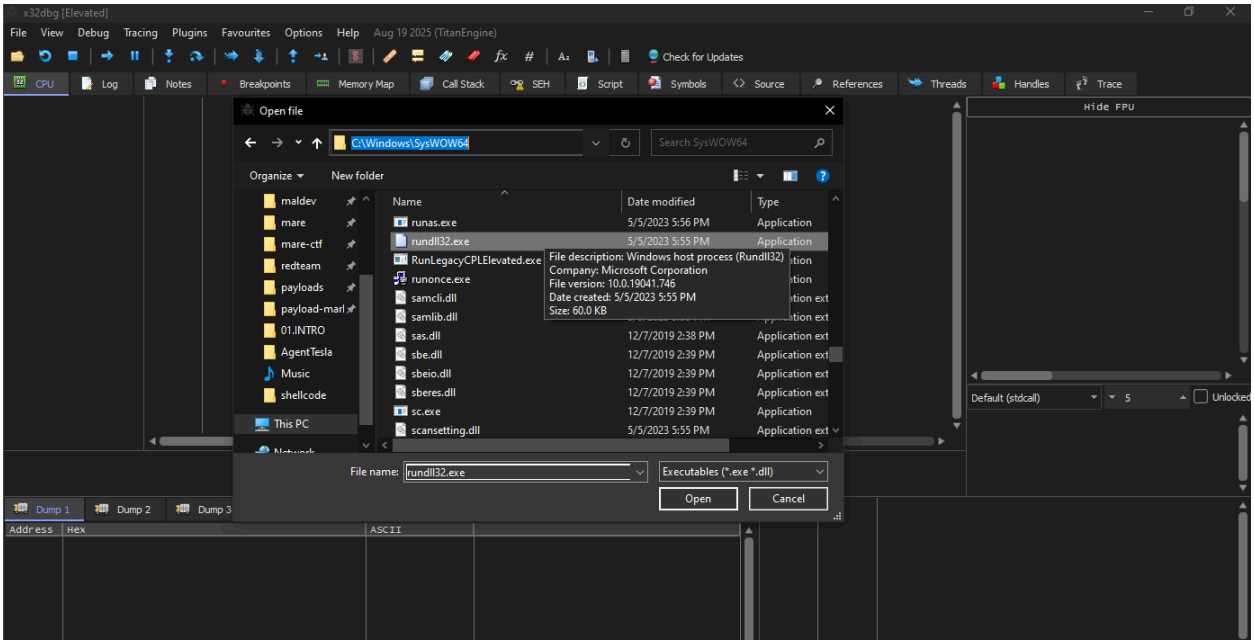

Fig 5 - loading rundll32 in x32dbg

update the command line

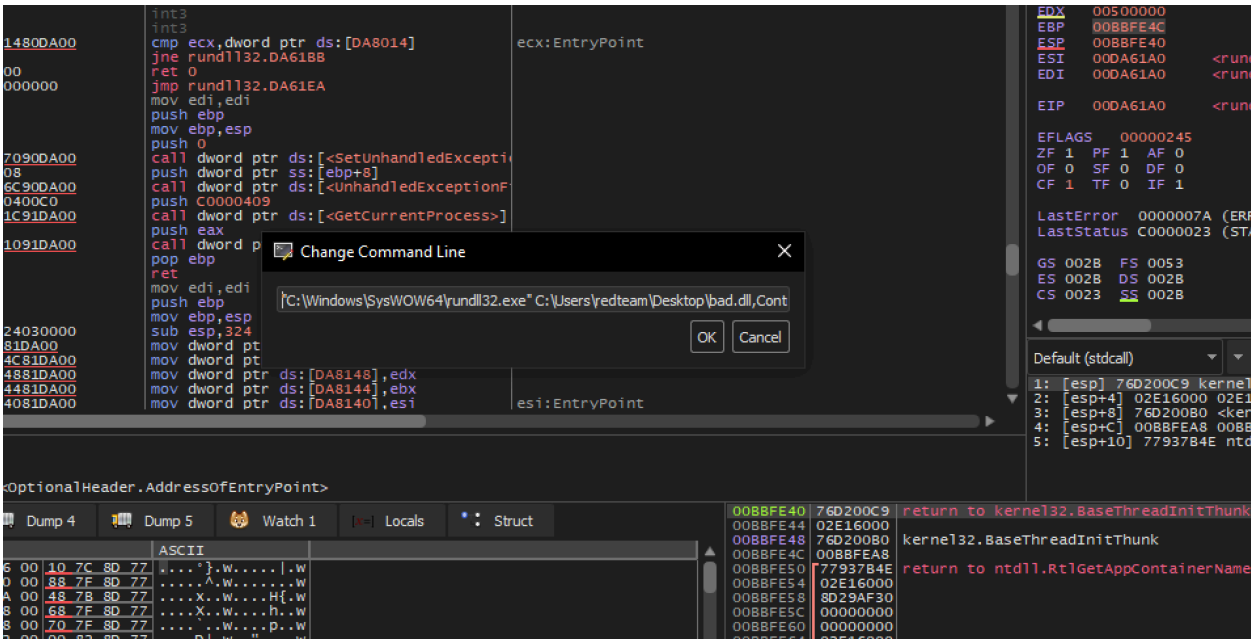

Fig 6 - updating command line

```
"C:\Windows\SysWOW64\rundll32.exe" C:\Users\redteam\Desktop\bad.dll,Control_RunDLL
```

Then check DLL ENTRY

option → Preference

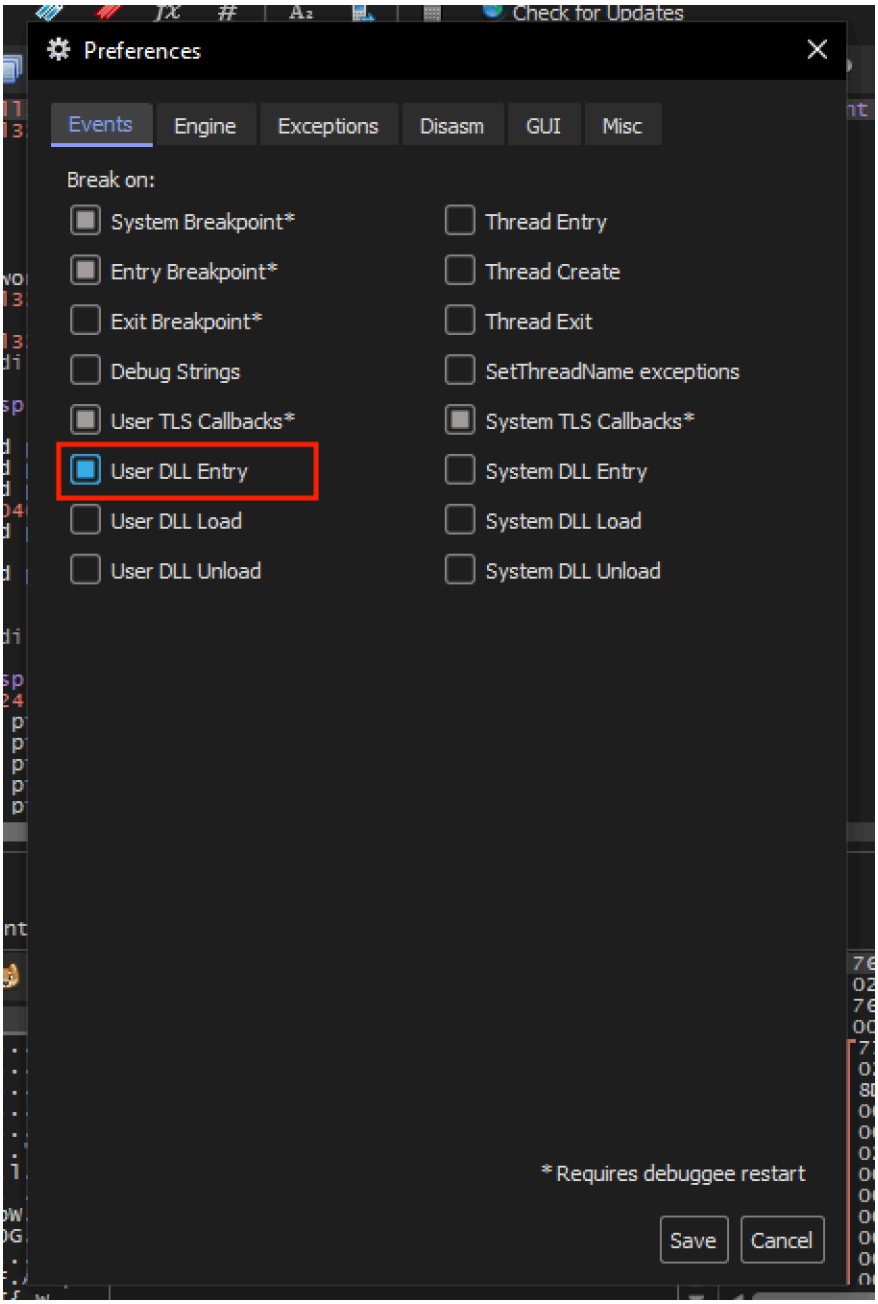

Fig 7 - Check dll entry

Then Debug → Restart

arrived at bad.dll

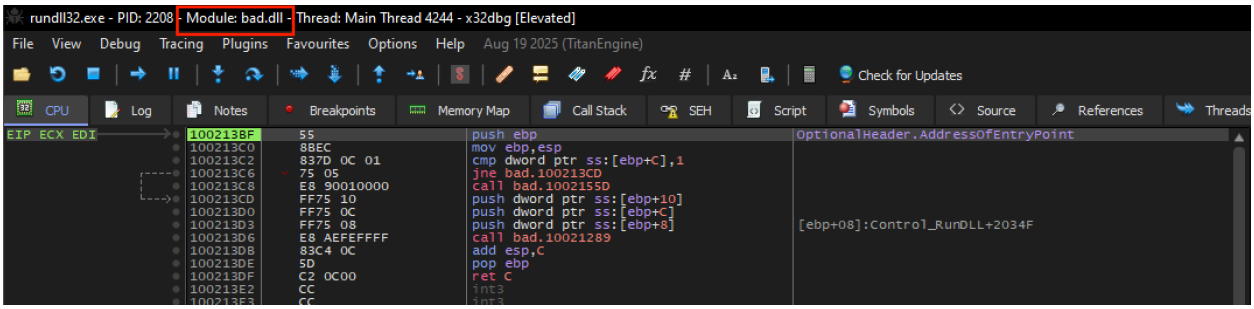

Fig 8 - bad.dll

Set breakpoint at `Control_RunDLL`

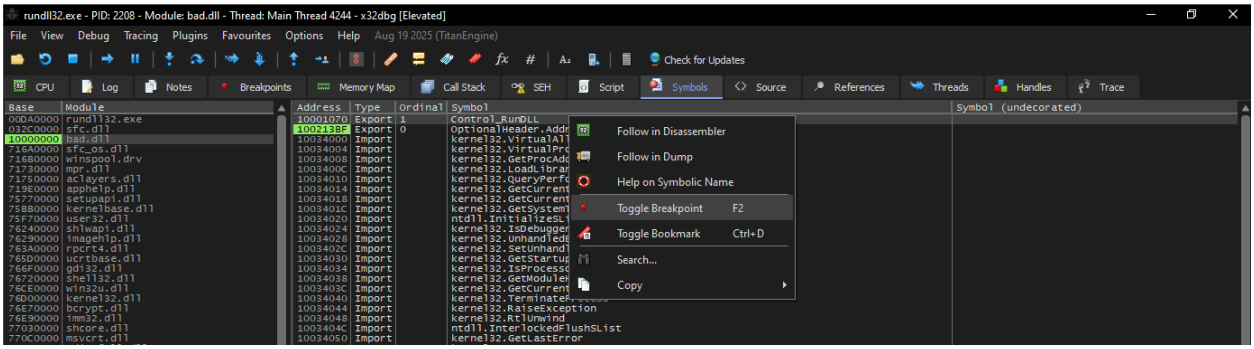

Fig 9 - breakpoint at `Control_RunDLL`

Arrived at `Control_RunDLL`

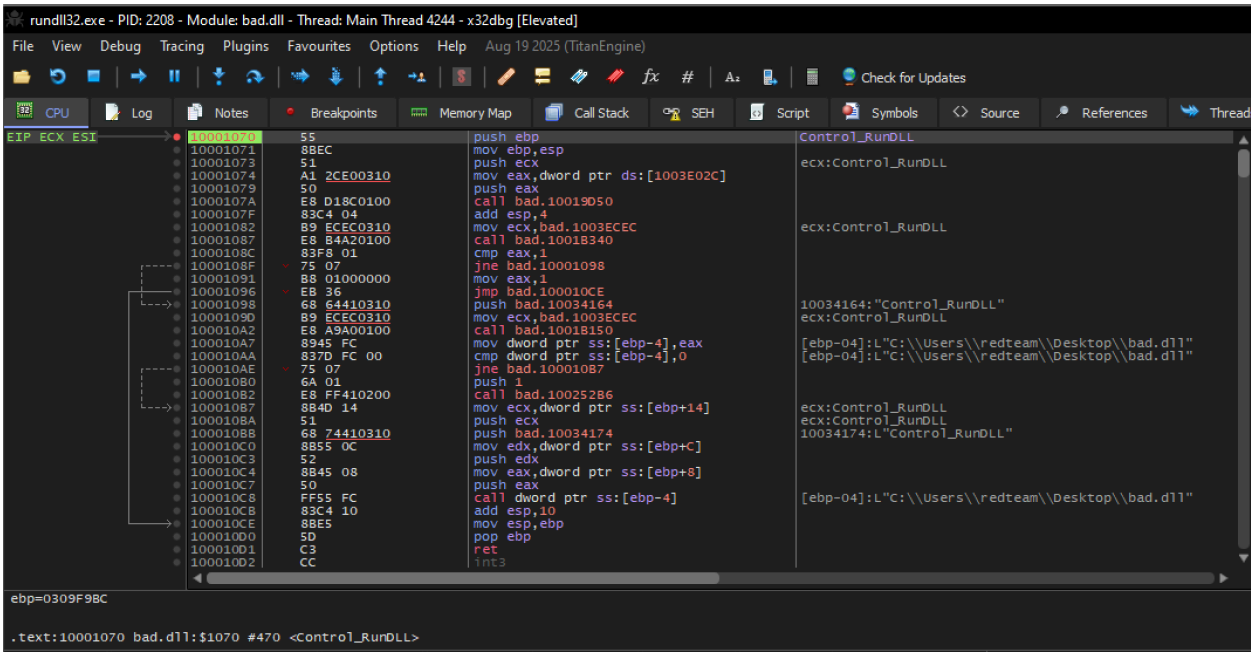

Fig 10 - `Control_RunDLL`

Setting breakpoint at the MZ header checking

In ghidra the comparison take place here.

```
10019ea4 0f b7 11        MOVZX      EDX,word ptr [this]
10019ea7 81 fa 4d        CMP        EDX,0x5a4d
                 5a 00 00
```

we need to inspect the address of `10019ea4` **This is where EDX is populated.**

set break point in x32 dbg

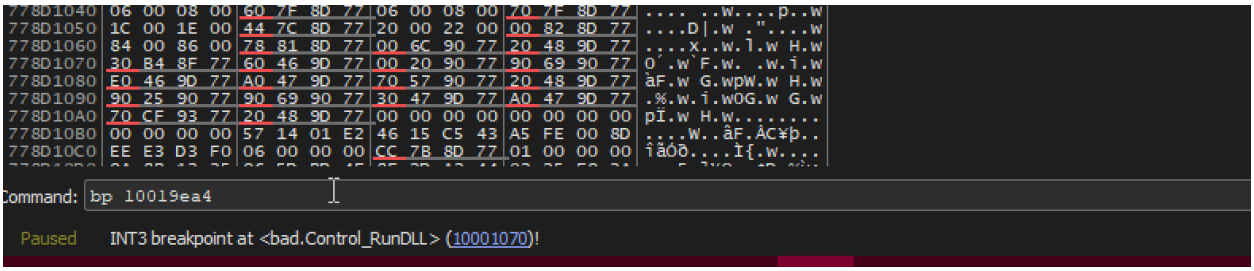

Fig 11 - breakpoint at 10019ea4

Continue running the program arrive at the break point `Follow ECX` **in memory dump.**

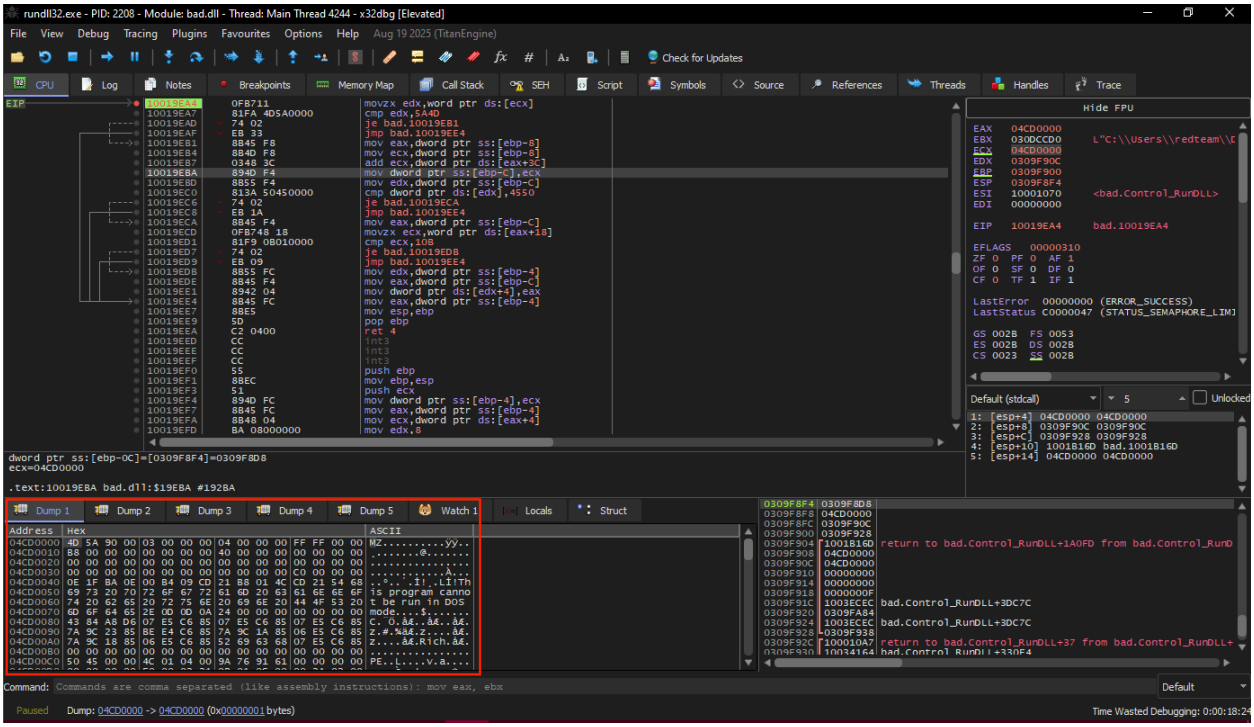

Fig 12 - MZ header

Follow in Memory Map

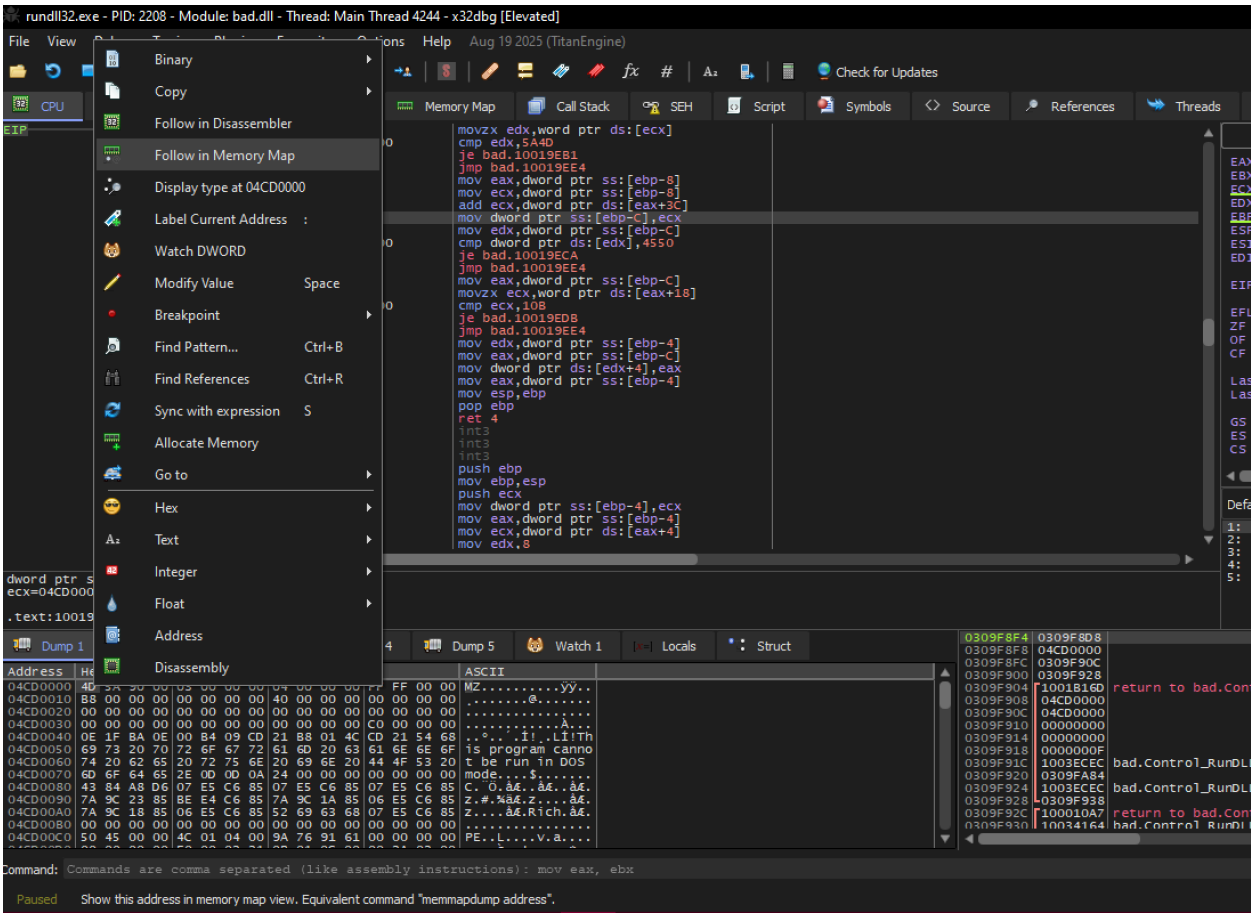

Fig 13 - follow in memory map

Dump Memory to file

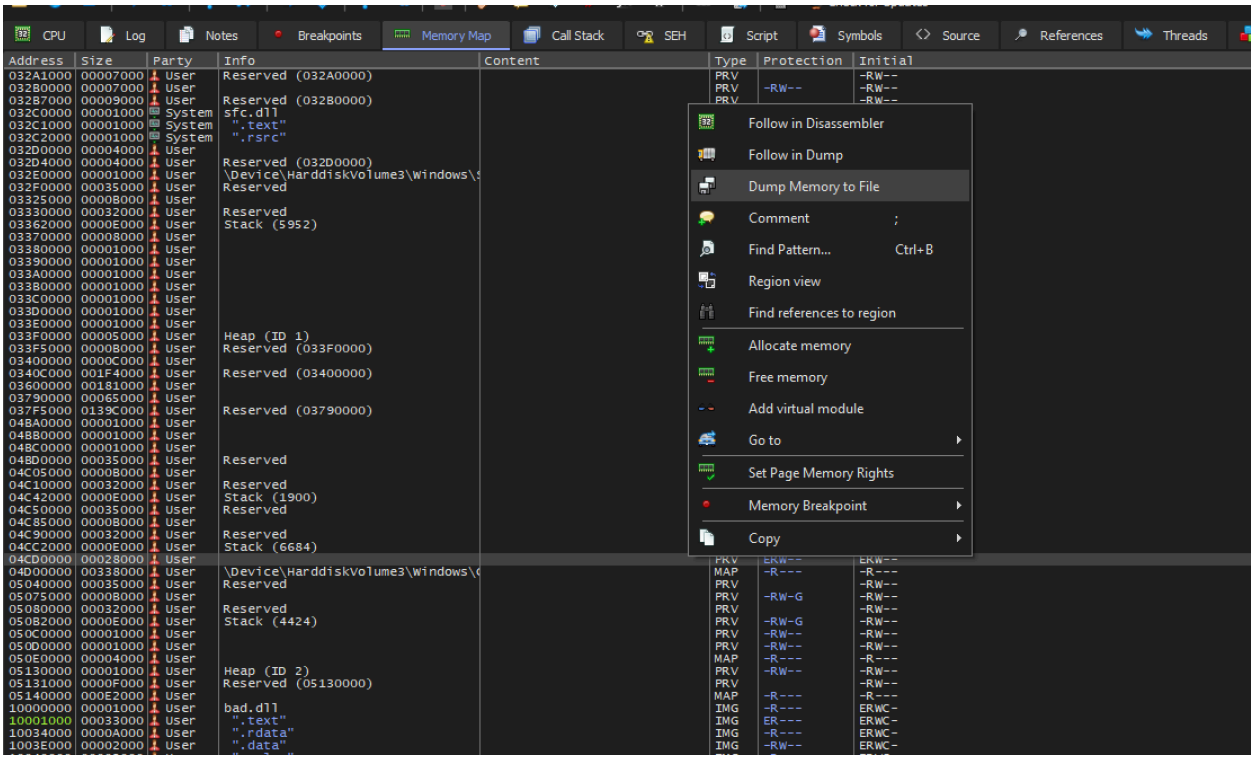

Fig 14 - Dump Memory to file

Save the dumped file

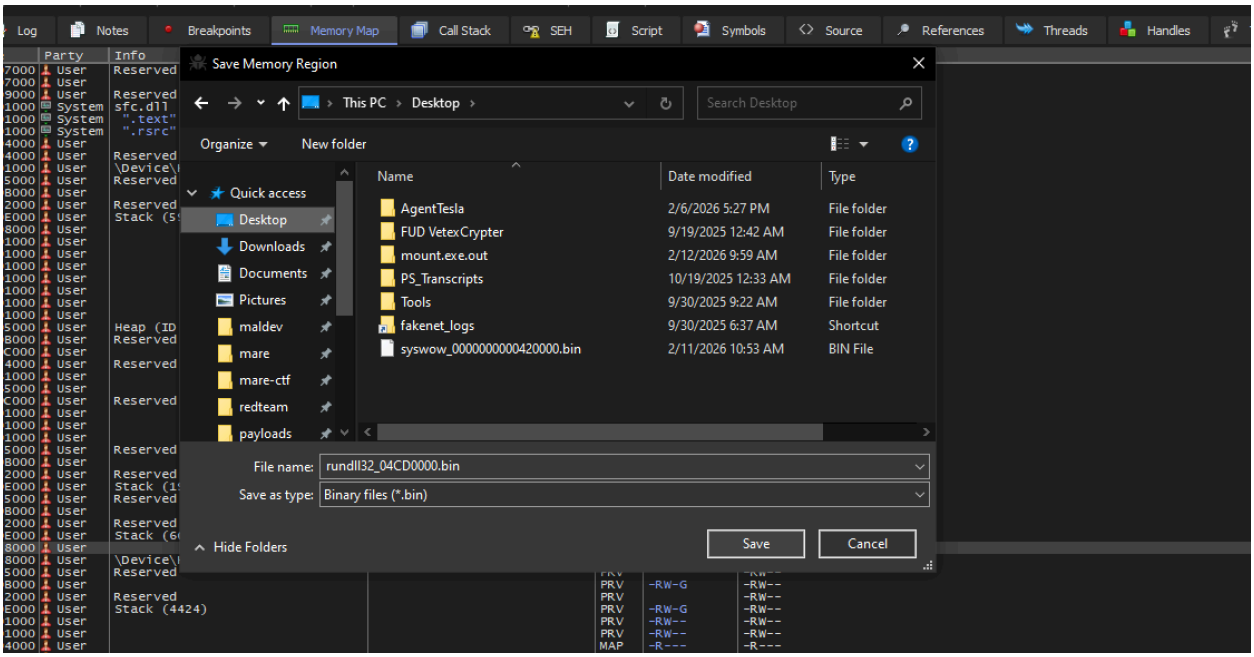

Fig 15 - saving dumped file

Analyzing rundll32_04CD0000.bin Stage 2

Payload meta data

Property	Value
File Type	Win32 DLL (PE32 executable, GUI, Intel 80386)
File Size	160.00 KB (163,840 bytes)
MD5	`e6e8784364adbfd04dc88dd0a3307063`
SHA-1	`db3f6755e41b934b2859994de538fd202f4c89fc`
SHA-256	`38e584a22754b9d84520e7db6f8853d51c120169f6e228e2a9d3c41fc8d513fc`
Vhash	`115046655d555"z`
Authentihash	`be806347d0cb5d2484ecae609ba861df184b7e6640f0a62fc579f92855cb9f92`
Rich PE Header Hash	`edcb0cf9819d81e55e04cf951af36ba4`
SSDEEP	`3072:Il2JsCxHcxJGPP5htzNhHGkEHy+/r8yFh60nMzJM6FKQ:hJsCuJEPlHQHye3nEFK`
TLSH	`T1D7F34A01F78381F7DC960CF219B6B22ECB7D1E037034EEA587990F57ADB5645A2A980D`
Magic	PE32 executable (DLL) (GUI) Intel 80386, for MS Windows

Payload PE section

Property	.text	.rdata	.data	.reloc
Entropy	**6.501**	5.275	5.651	5.470
File Ratio	86.56 %	0.31 %	0.63 %	0.63 %
Raw Address (Start)	`0x00000400`	`0x00022E00`	`0x00023000`	`0x00023400`
Raw Address (End)	`0x00022E00`	`0x00023000`	`0x00023400`	`0x00023800`
Raw Size	141,824 bytes	512 bytes	1,024 bytes	1,024 bytes
Virtual Address	`0x00001000`	`0x00024000`	`0x00025000`	`0x00027000`
Virtual Size	141,568 bytes	87 bytes	4,620 bytes	620 bytes

Ghidra Analysis

Key findings:

- **No static imports** — APIs resolved by walking PEB and hashing module/export names.
- **Heavy obfuscation** — control-flow flattening (state machine), XOR-encrypted strings and blobs, hashed API resolution.
- **Loader behavior** — loads one DLL from decrypted name, gets a function pointer, decrypts two blobs into a context structure.
- **Synchronization** — uses critical section–style APIs and a global flag; supports multi-step or coordinated execution.
- **Clean exit** — performs cleanup and returns; designed for one-shot execution (e.g. process injection).
- **Impact:** The shellcode itself does not perform direct malicious actions (e.g. file encryption or network C2) in the analyzed code. The primary malicious payload is expected to reside in the **loaded DLL** or in the **decrypted blobs**. Classification and risk depend on that payload and on the delivery mechanism.

Classification

Field	Value
Type	Shellcode / Loader
Platform	Windows (x86/x64; analysis used shellcode base)
Persistence	None in shellcode (single execution path)
Packed / Obfuscated	Yes — state machine, XOR encryption, API hashing
Capabilities observed	Dynamic API resolution, DLL loading, blob decryption, synchronization, controlled exit

Shellcode Functions Explained

Each section below shows the **decompiled C** for a key function and a short **explanation** of what it does.

1. `get_PEB` — Get Process Environment Block

```
void * get_PEB(void)
{
  return ProcessEnvironmentBlock;
}
```

What it does: Returns the **Process Environment Block (PEB)**. On x86 Windows the PEB is usually at `fs:[0x30]`. The shellcode has no imports, so it uses the PEB to find `kernel32.dll` / `ntdll.dll` and then walk the loader's module list and export tables to resolve APIs at runtime.

2. `hash_string_export` — Hash an ANSI string (for export names)

```
int __cdecl hash_string_export(undefined4 param_1, char *param_2)
{
  int iVar1;
  char cVar2;

  FUN_1000e901();
  iVar1 = 0;
  cVar2 = *param_2;
  while (cVar2 != '\\0') {
    iVar1 = (int)*param_2 + iVar1 * 0x1003f;
    param_2 = param_2 + 1;
    cVar2 = *param_2;
  }
  return iVar1;
}
```

What it does: Computes a **custom hash** of a null-terminated ANSI string: `hash = hash * 0x1003f + (unsigned char)*ptr`. Used when walking a DLL's **export table**: the malware hashes each export name and compares it to a stored hash (XOR'd with `0x1e5c48de`) to find the right function without storing the name in plain text.

3. `hash_module_name_wide` — Hash a wide (Unicode) module name

```
int hash_module_name_wide(void)
{
  int iVar1;
  ushort uVar2;
  uint uVar3;
  ushort *extraout_EDX;   /* caller passes wide string in EDX */
  ushort *puVar4;

  FUN_1000e901();
  iVar1 = 0;
  uVar2 = *extraout_EDX;
  puVar4 = extraout_EDX;
  while (uVar2 != 0) {
    uVar3 = (uint)*puVar4;
    if ((0x40 < uVar3) && (uVar3 < 0x5b)) {
      uVar3 = uVar3 + 0x20;   /* A–Z → a–z */
```

```
    }
    puVar4 = puVar4 + 1;
    iVar1 = uVar3 + iVar1 * 0x1003f;
    uVar2 = *puVar4;
  }
  return iVar1;
}
```

What it does: Same algorithm as above (`h = h * 0x1003f + char`), but on a **wide string**. Uppercase letters (0x41–0x5A) are lowercased (+0x20) so the hash is **case-insensitive**. Used when walking the PEB loader's module list: the malware hashes each module's base name and compares to a stored hash (XOR'd with `0x326e19fc`) to find e.g. `kernel32.dll` without a string in the binary.

---

4. `get_module_by_hash` — Find a loaded module by name hash

```
undefined4 __fastcall get_module_by_hash(undefined4 param_1, uint param_2)
{
  void *pvVar1;
  uint uVar2;
  undefined4 *puVar3;
  undefined4 *puVar4;

  pvVar1 = get_PEB();
  puVar4 = (undefined4 *)(*(int *)((int)pvVar1 + 0xc) + 0xc);   /* PEB->Ldr->InLoadOrderModuleLi
st */
  puVar3 = (undefined4 *)*puVar4;
  while (true) {
    if (puVar3 == puVar4) {
      return 0;
    }
    uVar2 = hash_module_name_wide();   /* hashes current module's BaseDllName */
    if ((uVar2 ^ 0x326e19fc) == param_2)
      break;
    puVar3 = (undefined4 *)*puVar3;   /* Flink = next module */
  }
  return puVar3[6];   /* DllBase (module base address) */
}
```

What it does: Walks **PEB → Ldr → InLoadOrderModuleList** (doubly linked list of loaded modules). For each node it hashes the module's base name with `hash_module_name_wide` and compares `hash ^ 0x326e19fc` to `param_2`. When it matches, it returns **DllBase** (the module's base address). So the malware finds e.g. `kernel32.dll` by a single DWORD hash.

---

5. `get_proc_by_hash` — Find an export by name hash

```
char * __fastcall get_proc_by_hash(undefined4 param_1, int param_2, undefined4 param_3,
                                   undefined4 param_4, uint param_5)
{
  int iVar1;
  int iVar2;
  int iVar3;
  int iVar4;
  uint uVar5;
  char *pcVar6;
  char *pcVar7;
  uint uVar8;
```

```
  FUN_1000e901();
  uVar8 = 0;
  iVar1 = *(int *)(param_2 + 0x3c);                    /* e_lfanew (PE header RVA) */
  pcVar6 = (char *)(*(int *)(iVar1 + 0x78 + param_2) + param_2);  /* Export directory */
  iVar2 = *(int *)(pcVar6 + 0x1c);   /* AddressOfFunctions */
  iVar3 = *(int *)(pcVar6 + 0x20);   /* AddressOfNames */
  iVar4 = *(int *)(pcVar6 + 0x24);   /* AddressOfNameOrdinals */
  while (true) {
    if (*(uint *)(pcVar6 + 0x18) <= uVar8)   /* NumberOfNames */
      return (char *)0x0;
    uVar5 = hash_string_export(0x11e71,
                (char *)(*(int *)(iVar3 + param_2 + uVar8 * 4) + param_2));  /* name at index uV
ar8 */
    if ((uVar5 ^ 0x1e5c48de) == param_5)
      break;
    uVar8 = uVar8 + 1;
  }
  pcVar7 = (char *)(*(int *)(iVar2 + param_2 + (uint)*(ushort *)(iVar4 + param_2 + uVar8 * 2) *
4)
                   + param_2);   /* RVA of function → absolute address */
  /* bounds check that RVA is inside export section */
  if (pcVar7 < pcVar6) {
    return pcVar7;
  }
  if (pcVar6 + *(int *)(iVar1 + 0x7c + param_2) <= pcVar7) {
    return pcVar7;
  }
  pcVar6 = (char *)FUN_10022c81(pcVar7);   /* resolve forwarder if needed */
  return pcVar6;
}
```

What it does: Given a **module base** (`param_2`), parses the PE **export directory** (NumberOfNames, AddressOfNames, AddressOfNameOrdinals, AddressOfFunctions). It hashes each export name with `hash_string_export` and compares `hash ^ 0x1e5c48de` to `param_5`. On match it computes the function address from the RVA and returns it (with optional forwarder resolution). So the malware gets e.g. `LoadLibraryW` or `GetProcAddress` by a single DWORD hash.

---

6. API resolver (cached) — `FUN_1001d9a7`

```
undefined4 __cdecl FUN_1001d9a7(int param_1, undefined4 param_2, uint param_3,
                                undefined4 param_4, uint param_5)
{
  int iVar1;
  char *pcVar2;

  /* param_1 = slot index, param_3 = module hash, param_5 = export hash */
  if (*(int *)(param_1 * 4 + 0x4cf5208) == 0) {
    iVar1 = get_module_by_hash(0x77, param_3);
    pcVar2 = get_proc_by_hash(0x650d9, iVar1, 0x116b, 0x92e5b, param_5);
    *(char **)(param_1 * 4 + 0x4cf5208) = pcVar2;
  }
  return *(undefined4 *)(param_1 * 4 + 0x4cf5208);
}
```

What it does: Resolves an API by **slot number** and **hashes** (module hash `param_3`, export hash `param_5`). The result is stored in a table at `0x4cf5208` (one pointer per slot). On first use it calls `get_module_by_hash` then `get_proc_by_hash` and caches the pointer; on later uses it just returns the cached value. So the shellcode never stores API names in plain text and only resolves each API once.

---

7. Heap allocator wrapper — `FUN_10022e91`

```
void __fastcall FUN_10022e91(undefined4 param_1, undefined4 param_2, undefined4 param_3,
                             undefined4 param_4, undefined4 param_5, undefined4 param_6)
{
  code *pcVar1;

  FUN_1000e901();
  pcVar1 = (code *)FUN_1001d9a7(0x2a, 0x44, 0xbd10ff8e, 0x44, 0xba3519d7);  /* resolve slot 0x2a
*/
  (*pcVar1)(param_2, param_6, param_5);   /* (heap_handle?, size, alignment) → RtlAllocateHeap s
tyle */
  return;
}
```

What it does: Resolves the API in **slot 0x2a** (hash `0xba3519d7` — typically **RtlAllocateHeap** or similar) and calls it with `(param_2, param_6, param_5)`, i.e. heap handle, alignment, size. So this is the shellcode's **heap allocation** path (used by the decrypt and other routines that need memory).

---

8. `decrypt_wide_string` — XOR-decrypt a wide string

```
ushort * __cdecl decrypt_wide_string(undefined4 param_1, uint *param_2)
{
  uint *puVar1;
  uint uVar2;
  uint uVar3;
  ushort *puVar4;
  uint uVar5;
  ushort *puVar6;
  uint uVar7;
  uint uVar8;
  uint *puVar9;

  FUN_1000e901();
  uVar2 = *param_2;              /* key[0] */
  puVar9 = param_2 + 2;           /* encrypted data after key */
  uVar3 = param_2[1] ^ uVar2;     /* length (stored XOR'd with key) */
  uVar7 = uVar3 + 1;
  uVar8 = uVar7;
  if ((uVar7 & 3) != 0) {
    uVar8 = (uVar7 & 0xfffffffc) + 4;   /* round up to multiple of 4 DWORDs */
  }
  puVar4 = (ushort *)FUN_10016610(uVar7 & 0xffffff03, uVar8 * 2);   /* allocate uVar8 * 2 bytes
*/
  if (puVar4 != (ushort *)0x0) {
    uVar7 = 0;
    puVar1 = (uint *)((int)puVar9 + (uVar8 & 0xfffffffc));
    uVar8 = (uint)((int)puVar1 + (3 - (int)puVar9)) >> 2;
    if (puVar1 < puVar9) {
      uVar8 = 0;
```

```
    }
    puVar6 = puVar4;
    if (uVar8 != 0) {
      do {
        uVar5 = *puVar9;
        puVar9 = puVar9 + 1;
        uVar5 = uVar5 ^ uVar2;    /* XOR with key */
        *puVar6 = (ushort)uVar5 & 0xff;
        puVar6[1] = (ushort)(uVar5 >> 8) & 0xff;
        uVar7 = uVar7 + 1;
        puVar6[2] = (ushort)(uVar5 >> 0x10) & 0xff;
        puVar6[3] = (ushort)(byte)(uVar5 >> 0x18);
        puVar6 = puVar6 + 4;
      } while (uVar7 < uVar8);
    }
    puVar4[uVar3] = 0;   /* null-terminate wide string */
  }
  return puVar4;
}
```

What it does: Decrypts a **XOR-encrypted wide string**. Layout at `param_2`: two DWORDs (key), then DWORDs of ciphertext. Length is recovered as `param_2[1] ^ param_2[0]`. It allocates memory, XORs each DWORD with the key, writes the result as wide chars, and null-terminates. Used for DLL names and other Unicode strings embedded in the shellcode.

---

9. `decrypt_and_alloc_blob` — XOR-decrypt a binary blob

```
int __cdecl decrypt_and_alloc_blob(undefined4 param_1, undefined4 param_2, uint *param_3,
                                   uint *param_4)
{
  uint *puVar1;
  uint uVar2;
  int iVar3;
  int iVar4;
  uint uVar5;
  uint uVar6;
  uint *puVar7;
  uint uVar8;

  FUN_1000e901();
  uVar8 = 0;
  uVar2 = *param_3;              /* key */
  puVar7 = param_3 + 2;          /* encrypted blob */
  uVar5 = param_3[1] ^ uVar2;    /* size in DWORDs */
  uVar6 = uVar5;
  if ((uVar5 & 3) != 0) {
    uVar6 = (uVar5 & 0xfffffffc) + 4;
  }
  iVar3 = FUN_10016610(uVar5 & 0xffffff03, uVar6);   /* allocate */
  if (iVar3 != 0) {
    puVar1 = (uint *)((int)puVar7 + (uVar6 & 0xfffffffc));
    uVar6 = (uint)((int)puVar1 + (3 - (int)puVar7)) >> 2;
    if (puVar1 < puVar7) {
      uVar6 = 0;
    }
    if (uVar6 != 0) {
```

```
      iVar4 = iVar3 - (int)puVar7;
      do {
        uVar8 = uVar8 + 1;
        *(uint *)(iVar4 + (int)puVar7) = *puVar7 ^ uVar2;   /* XOR with key */
        puVar7 = puVar7 + 1;
      } while (uVar8 < uVar6);
    }
    if (param_4 != (uint *)0x0) {
      *param_4 = uVar5;   /* optional: return decrypted size */
    }
  }
  return iVar3;   /* pointer to decrypted buffer or 0 */
}
```

What it does: Same idea as above but for **raw DWORD blobs**: key in first two DWORDs, length as `param_3[1] ^ param_3[0]`, then encrypted data. Allocates, XOR-decrypts in place, and optionally writes the length to `param_4`. Used for the two large blobs at `0x4cd1000` and `0x4cd1060` (payload/config).

---

10. `get_global_flag_0x220` — Read global state flag

```
undefined4 get_global_flag_0x220(void)
{
  return *(undefined4 *)(_DAT_04cf61fc + 0x220);
}
```

What it does: Reads a single **DWORD** at offset `+0x220` in the global context structure at `_DAT_04cf61fc`. The main state machine uses this as a **flag** (e.g. "initialization done" or "error") to decide the next state. No arguments; pure read of shared state.

---

11. `FUN_1001f413` — Get value in range from internal stream (PRNG-like)

```
int __cdecl FUN_1001f413(int param_1, int param_2)
{
  uint *puVar1;
  uint uVar2;
  undefined4 *puVar3;

  puVar3 = _DAT_04cf61f4;
  if ((uint)_DAT_04cf61f4[7] <= _DAT_04cf61f4[4] + 4) {
    _DAT_04cf61f4[4] = *_DAT_04cf61f4;   /* wrap cursor to start of buffer */
  }
  puVar1 = (uint *)puVar3[4];
  uVar2 = *puVar1;
  puVar3[4] = puVar1 + 1;   /* advance cursor */
  return uVar2 % (uint)(param_1 - param_2) + param_2;
}
```

What it does: Uses a global buffer at `_DAT_04cf61f4`: `[0]` = base, `[4]` = current pointer, `[7]` = end. It reads the next DWORD, advances the cursor (wrapping to the start if past the end), and returns that value **modulo (param_1 - param_2)** plus **param_2**, i.e. a value in the range `[param_2, param_1)`. So it behaves like a **PRNG or decode stream** producing numbers in a given range (e.g. 4000–8000 or 30000–60000), used for delays or branching in the state machine.

---

12. `entry` — Main state machine (excerpt)

The full `entry` is a long if/else chain; below is the **pattern** and a **short excerpt** showing how state drives the loader.

Pattern: One variable (e.g. `unaff_ESI`) holds the current state. The code loops to `LAB_10021cf3`, compares the state to constants, calls a helper, sets the next state, and jumps back.

```
void entry(void)
{
  /* ... many stack/register vars ... */
  int unaff_ESI;   /* current state */

  /* constants used later (e.g. ranges for FUN_1001f413) */
  uStack000000d4 = 10000;
  iStack0000012c = 4000;
  iStack000001b8 = 8000;
  /* ... */

LAB_10021cf3:
  iVar4 = unaff_ESI;
  if (iVar4 == 0x7b75f67) {
    iVar4 = FUN_10005010();      /* init check */
    if (iVar4 == 0) return;
    unaff_ESI = 0x86b14cd;
    goto LAB_10021cf3;
  }
  /* ... */
  if (iVar4 == 0x42cce25) {
    bVar1 = FUN_1001bc74();      /* alloc API table */
    if (CONCAT31(extraout_var, bVar1) == 0) return;
    unaff_ESI = 0x2e8c6c1;
    goto LAB_10021cf3;
  }
  if (iVar4 == 0x2e8c6c1) {
    FUN_1000c00e();              /* resolve 8 APIs into table */
    unaff_ESI = 0x335fb9b;
    goto LAB_10021cf3;
  }
  if (iVar4 == 0x335fb9b) {
    iVar4 = FUN_1000c695();      /* load DLL, GetProcAddress */
    if (iVar4 == 0) return;
    unaff_ESI = 0xee82d2a;
    goto LAB_10021cf3;
  }
  if (iVar4 == 0x2905ca9) {
    pvVar2 = (void *)decrypt_and_alloc_blob(..., (uint *)0x4cd1060, ...);
    pvVar3 = (void *)decrypt_and_alloc_blob(..., (uint *)0x4cd1000, ...);
    iVar4 = FUN_1001bcdd(...);   /* init context from decrypted blobs */
    FUN_10006fe1(..., pvVar3);
    FUN_10006fe1(..., pvVar2);
    /* next state 0x9e27a8 (return) or 0x52aa28e */
  }
  /* ... many more states ... */
  if (iVar4 == 0x2a1ede3) {
    FUN_1001b214();   /* cleanup */
    return;
  }
  /* ... */
```

```
  goto LAB_10021cf3;
}
```

What it does: entry is the shellcode's only exported function. It implements a **big state machine**: each state (e.g. `0x7b75f67`, `0x42cce25`, `0x2e8c6c1`, `0x335fb9b`, `0x2905ca9`, `0x2a1ede3`) runs one step (init, alloc API table, resolve 8 APIs, load DLL, decrypt two blobs and init context, ... cleanup) and then sets the next state and jumps back to the top. This **control-flow flattening** makes the flow harder to follow in a disassembler. The initial value of `unaff_ESI` (or equivalent) is set by the caller (e.g. injector) and determines which path is taken first.

Quick reference

Function	Role
`get_PEB`	Return PEB (for API resolution).
`hash_string_export`	Hash ANSI string (export names).
`hash_module_name_wide`	Hash wide string, case-insensitive (module names).
`get_module_by_hash`	Find loaded module by name hash (walk PEB Ldr list).
`get_proc_by_hash`	Find export by name hash (walk PE export dir).
`FUN_1001d9a7`	Resolve API by slot + hashes; cache at `0x4cf5208`.
`FUN_10022e91`	Allocate heap block (slot 0x2a → RtlAllocateHeap-style).
`decrypt_wide_string`	XOR-decrypt wide string; allocate and return.
`decrypt_and_alloc_blob`	XOR-decrypt DWORD blob; allocate and return.
`get_global_flag_0x220`	Read global flag at context+0x220.
`FUN_1001f413`	Next value in range [param_2, param_1) from stream.
`entry`	Main state machine: init → resolve APIs → load DLL → decrypt blobs → ... → cleanup.

3. Indicators of Compromise (IOCs)

3.1 File / Memory

Description	Value / Location
Entry point (export)	`entry` at offset `0x10021a20` (adjust for actual base)
API pointer cache	Table at relative offset corresponding to `0x4cf5208` (slot-indexed)
Encrypted blobs	Two blobs at offsets corresponding to `0x4cd1000`, `0x4cd1060`
Encrypted DLL name	String at offset corresponding to `0x4cd192c`
Encrypted API-name strings	8 strings at offsets `0x4cd12e4`, `0x4cd1304`, `0x4cd1324`, `0x4cd1344`, `0x4cd1364`, `0x4cd1384`, `0x4cd13b4`, `0x4cd1424`
Global context	Structures at offsets corresponding to `0x4cf61fc`, `0x4cf61f4`, `0x4cf61f8`, `0x4cf6200`, `0x4cf61f0`

Note: Addresses are relative to the base address used in the analyzed dump. In a live or different load, rebase these or express as offsets from the shellcode base.

Technical Analysis

Entry Point and Control Flow

- **Entry:** Single exported symbol `entry` at `0x10021a20`.
- **Structure:** The entry function is a **state machine dispatcher**. The current state is held in a register (e.g. ESI) or equivalent. Each state is a large numeric constant (e.g. `0x7b75f67`, `0x2905ca9`, `0x2a1ede3`). The code loops, compares the state, calls a subroutine, and sets the next state or returns.

- **Subroutines:** Many subroutines are also implemented as state machines (control-flow flattening), making static analysis harder.

4.2 API Resolution

- **Method:** No import table. APIs are resolved at runtime:
  1. **Module resolution:** Walk PEB → `Ldr` → `InLoadOrderModuleList`; hash each module's base name (wide string, case-insensitive) with algorithm `h = h * 0x1003f + char`; compare to a stored hash (XOR constant `0x326e19fc`).
  2. **Export resolution:** Given a module base, parse PE export directory; hash each export name (ANSI) with the same multiplier `0x1003f`; compare to a stored hash (XOR constant `0x1e5c48de`).
- **Caching:** Resolved function pointers are stored in a table (base at offset corresponding to `0x4cf5208`), indexed by a slot number (e.g. `0x2a`, `0x7c`, `0x32a`, `0x3ae`, `0x260`).
- **Likely APIs (by slot/hash usage):** Heap/virtual alloc (e.g. `RtlAllocateHeap`, `NtAllocateVirtualMemory`), `LoadLibrary`/`LoadLibraryEx`, `GetProcAddress`, and critical section initialization (e.g. `RtlInitializeCriticalSection`).

Encryption and Decryption

- **Strings:** Wide-character strings are stored as XOR-encrypted data with a 2-DWORD key. A dedicated function decrypts them, allocates memory, and returns the wide string. Used for DLL names and API names.
- **Blobs:** Two larger blobs (at offsets corresponding to `0x4cd1000` and `0x4cd1060`) are XOR-decrypted and copied into allocated memory; the result is used to initialize a context structure (`_DAT_04cf61f8`). These may contain configuration or secondary payload.

Loader Logic

1. Allocate a 0x220-byte block (early global init).
2. Allocate a 0x44-byte table and resolve **8** API pointers by decrypting 8 strings and resolving each (stored at `_DAT_04cf6200 + 8 + index*4`).
3. Allocate a 0x30-byte structure and a 0x4000-byte buffer; decrypt the DLL name from `0x4cd192c`.
4. Load the DLL (LoadLibrary-style) and resolve at least one export (GetProcAddress-style).
5. Decrypt the two blobs and initialize the context structure; continue the state machine (e.g. PRNG-like stream, sync, checks).
6. Run the state machine until a cleanup state (`0x2a1ede3`); call cleanup (e.g. free resources, zero); return.

Memory and Globals

- **Allocation:** Wrappers call virtual alloc– and heap alloc–like APIs (by hash). Allocation size and alignment (e.g. 8) are passed as arguments.
- **PRNG-like stream:** A global buffer (`_DAT_04cf61f4`) is used as a stream; a helper returns a value in a range `[param_2, param_1)` (e.g. 4000–8000, 60000–30000), possibly for delays or branching.
- **Global flag:** A DWORD at offset `+0x220` in the main context (`_DAT_04cf61fc`) is read in multiple states and influences the next state (e.g. error handling or phase).

Synchronization

- Critical section–style initialization (slot `0x32a`, hash `0x75d5dc06`) is called with `(alloc_handle, 0, context_ptr)`. The same pattern is used when initializing per-structure context (e.g. before using the decrypted blobs), indicating multi-step or thread-aware design.

Behavior Summary

Phase	Action
Init	Allocate 0x220-byte global block; allocate API table (0x44 bytes).

Phase	Action
API resolution	Resolve 8 APIs into table; resolve additional APIs by hash into slot table (e.g. alloc, LoadLibrary, GetProcAddress, critical section).
DLL load	Decrypt DLL name → LoadLibrary → GetProcAddress (at least one export).
Payload setup	Decrypt two blobs; initialize context structure; initialize critical section.
Main loop	State machine uses PRNG-like stream, global flag, and sync; multiple branches and states.
Cleanup	Free resources; finalize; return.

The **primary malicious behavior** (e.g. ransomware, info stealer, C2) is expected to be in the **loaded DLL** or in code/data from the **decrypted blobs**, not in the shellcode's direct API calls.

Appendix

Key Function Mapping (Ghidra)

Analyzed name	Renamed / role
FUN_1001e9ca	get_PEB
FUN_1001931b	get_module_by_hash
FUN_100185a5	get_proc_by_hash
FUN_1001de14	hash_string_export
FUN_100177e7	hash_module_name_wide
FUN_100186f5	decrypt_wide_string
FUN_10016d89	decrypt_and_alloc_blob
FUN_10006142	get_global_flag_0x220
FUN_1001d9a7	API resolver (slot + hashes; cache at 0x4cf5208)

State Machine (abbreviated)

Initialization chain (typical path):

`0x7b75f67` → init check → `0x86b14cd` → … → `0x42cce25` → alloc API table → `0x2e8c6c1` → fill 8 APIs → `0x335fb9b` → load DLL → `0xee82d2a` → … → `0x2905ca9` → decrypt two blobs, init context → … → main loop states → `0x2a1ede3` → cleanup → return.

Exact path depends on initial state (e.g. value in ESI when shellcode is invoked).
